



Phase 1

Phase 1: Hybrid Reactivity Core

(3-4 weeks |   )

Milestone	Icon	Key Points	Portfolio Tip
Core Signals 		- <code>signal()</code> / <code>computed()</code> / <code>effect()</code> API- Explicit dependency tracking- Microtask batching	Write blog: "Why Signals > Observables"
Deep Reactivity Layer 		- Opt-in <code>deepSignal()</code> - Proxy-based nesting- Memory-safe with WeakMap	Demo: <i>Signal vs Proxy performance test</i>
Batching & Cleanup		- <code>batch()</code> for atomic updates- Context-based destruction- Zombie effect prevention	Video: <i>Effect execution timeline</i>
Debugging Tools		- Dev-mode dependency warnings- Signal graph visualization	Build: <i>Interactive dependency viewer</i>

Timeline

Copy

Week 1:  Core Signals
Week 2:  Deep Reactivity
Week 3:  Batching & Cleanup
Week 4:  Debugging Tools

Key Deliverables Preview

1. Code Example:

ts

Copy

```
const [count, set] = signal(0);
const user = deepSignal({ profile: { name: "Alice" } });
effect(() => console.log(count(), user.profile.name));
```

2. Portfolio Hook:

- "Hybrid Reactivity: When to Use Signals vs Proxies" guide
- Benchmark vs Angular/Vue (3x faster updates 🚀)

Design Principles

-  **Opt-In Complexity:** Start simple (Angular-style), add magic later
-  **Memory-Safe:** WeakMap + GC-friendly patterns
-  **Framework-Agnostic:** Isolate reactivity for future reuse

Key Takeaway: This phase builds a reactivity system that's *explicit like Angular*, *deep like Vue*, and *debuggable like Solid*.

▼ Description:

🌟 Phase 1: Building the Reactivity Nervous System

(Where Your Framework Comes Alive)

What We're Building

Imagine giving your framework a **digital nervous system** — wires that *feel* data changes and *ripple* updates exactly where needed. In Phase 1, we're crafting:

- ⚡ **Signals:** Atomic state units that scream "*I changed!*" when updated
- 🌀 **Deep Reactivity:** Proxies that auto-track nested objects like psychic detectives

- 🕒 **Glitch-Free Updates:** A scheduler preventing Frankenstein states during rapid changes
- 🧹 **Self-Cleaning Effects:** A janitor that vaporizes orphaned computations

This isn't just code — it's the **beating heart** of your framework's responsiveness.

Theoretical Challenges

(The Dragons We Slay Here)

1. The Glitch Paradox

- *Problem:* $A \rightarrow B \rightarrow C$ dependency chain could render intermediate garbage states
- *Solution:* Microtask batching + topological effect sorting

2. Memory Leak Quicksand

- *Problem:* Forgotten subscriptions slowly suffocate your app's performance
- *Solution:* WeakMap-based tracking + destroy-aware context trees

3. Zombie Effect Apocalypse

- *Problem:* Components die, but their effects keep haunting the runtime
- *Solution:* Component lifecycle-tied effect disposal (think “digital holy water”)

4. Dependency Hell

- *Problem:* Over-tracking turns simple `user.name` into a 50-node dependency graph
 - *Solution:* Opt-in deep reactivity + dev-mode tracking warnings
-

What You'll Master

By the end of this phase, you'll wield:

- **Reactive Sorcery:**

- Know when to use *push* (signals) vs *pull* (proxies) reactivity
 - Make 10,000 DOM updates dance at 60fps
 - **Memory Kung Fu:**
 - Design systems that self-clean like a Tesla's battery management
 - Weaponize WeakMap/WeakRef to fight memory leaks
 - **Scheduling Wizardry:**
 - Microtask queues vs RAF vs debounced updates
 - Prevent flicker-inducing update collisions
 - **Debugging X-Ray Vision:**
 - Visualize dependency graphs like Neo seeing the Matrix
 - Trace why *that effect* ran 37 times unexpectedly
-

Why This Matters

This phase transforms you from a framework *user* to a reactivity *architect*. You'll:

- **Internalize** how Angular/Vue/Solid reactivity truly work under the hood
 - **Discover** why 90% of framework bugs originate here (and how to avoid them)
 - **Build** something recruiters *viscerally understand*: "They made their own Reactivity engine?!"
-

Ready to make digital atoms dance to your tune?

This is where your framework stops being lines of code and starts *breathing*. Let's build the nervous system first — the muscles (components) and skeleton (DOM) will follow!

▼ Pillar:

 Signals

▼ **Files:**

Signal.ts